

AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 8.1: The Wall — Why Naive Prompting Fails

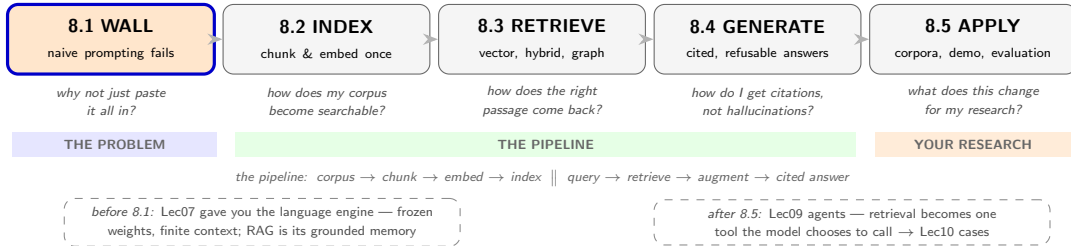
Zhigang Feng

2026-07-08 07:33:42

Lecture 8 in One Map

One lecture, five acts: hit the wall, build the machine that walks around it, then put it to research work.

you are here ▼



Route this deck: the frozen brain → four walls → a wiki by hand. Case threads: (A) the **FOMC corpus**, opened by the December-2025 question; (B) the **OLG literature** (157 papers), queried by graph in Act 8.3 and by hand in Act 8.5.

Where We Stand

The language engine: frozen weights, finite context

The Frozen Brain: What Lecture 7 Left Us

Lecture 7 left us a powerful reader whose knowledge is frozen: everything it “knows” lives in weights fixed at the end of pretraining.

- **The machine** (Lec. 7): tokens $\rightarrow$ embeddings $\rightarrow$ attention layers $\rightarrow$ a probability distribution over the next token. Knowledge is *parametric* — there is no memory beyond the weights and the current context window.
- **Knowledge cutoff**: every model’s training corpus stops months before you use it. And your *private* corpora — working papers in progress, internal memos, FOIA-released records — were never in anyone’s training data at all.

Lecture 8’s opening question (Lec. 7 T5a closed on it): “*What did the FOMC decide at its December 2025 meeting?*” A model trained before that meeting has *never seen* the statement. It will either refuse — or, worse, hallucinate a plausible but fictional answer.

Today’s question: how do we extend the model’s knowledge *at inference time*, without retraining it?

Three Pain Points for the Economist

For research use, the frozen brain fails in three specific ways.

- **1. Stale knowledge.** Anything published after the cutoff is invisible. So is anything proprietary. Macro and finance research runs on the *latest* releases.
- **2. Hallucinated citations.** Ask a vanilla LLM to “cite five NBER papers on the slope of the Phillips curve” and it will happily produce five plausible titles, plausible authors, and entirely fake DOIs. This is the most common failure mode in academic use.
- **3. No source attribution.** Even when the answer is correct, the model cannot tell you *where* the claim came from. Academic research requires footnotes.

Requirements for research-grade work: a system that can (i) read documents the model never saw, (ii) refuse to invent facts, and (iii) tell us which document supplied each claim.

Two Ways to Add Knowledge

There are exactly two places to put new knowledge: into the weights, or into the prompt.

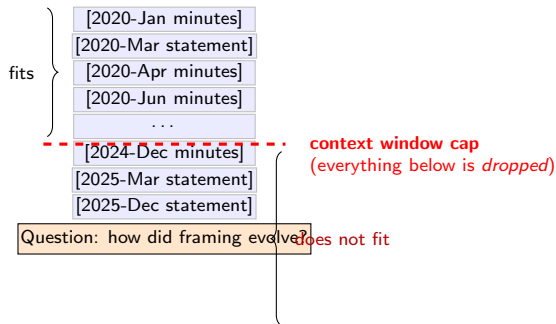
- **Option A — Fine-tuning:** re-train (some of) the model's weights on your new corpus.
 - Pros: knowledge becomes “baked in,” fast at inference.
 - Cons: expensive, fragile, hard to update, no source attribution, doesn't fix hallucination.
- **Option B — Retrieval (RAG):** leave the model untouched. At query time, fetch the relevant text from an external store and paste *only that* into the prompt.
 - Pros: cheap, updatable in real time, gives source attribution, dramatically reduces hallucination.
 - Cons: more moving parts; quality depends on the retriever.
- **This lecture's promise:** Acts 8.2–8.3 build the index and the retrieval; Act 8.4 turns retrieved evidence into a cited answer — and settles A vs. B with a decision matrix; Act 8.5 applies the pipeline to six research corpora.

The Wall

Four walls: the window, the middle, the meter, the memory

A Tempting First Attempt: Paste It All In

Setup (Lec. 7 T4 covered prompting itself — zero-shot, few-shot, in-context learning). The naive RAG-less workflow goes one step further: paste the *whole corpus* into the prompt. **Research question:** “How did the FOMC’s framing of inflation expectations evolve from 2020 to 2025?”



Four walls block this approach (toured in pairs next):

1. the **window** — context cap (red line)
2. the **middle** — “lost in the middle” (Lec. 7 T5a)
3. the **meter** — pay every token, every query
4. the **memory** — no persistence across queries

*In principle the model has all the data;
in practice all four walls bind.*

Hard Walls: The Window and the Middle

Wall 1 is a hard cap; wall 2 degrades reading even inside the cap.

- **The window.** Frontier models read $\sim 1\text{M}$ tokens at once (mid-2026); many deployed models cap far lower. Long corpora *force* chunking, summarization, or retrieval (Lec. 7 T2a).
- **How big is the FOMC corpus?** 8 meetings/year $\times$ [statement $\sim 1\text{K}$ + minutes $\sim 15\text{K}$ tokens] $\approx 130\text{K}/\text{year}$ *excluding* transcripts; five years (2020–2024) $\approx 650\text{K}$ — it *barely* fits, and only as the whole budget.
 - Transcripts run $\sim 100\text{K}$ *each* (5-year lag): one year approaches 1M alone. Thirty years of statements+minutes $\approx 3.9\text{M}$; add speeches, Beige Books, NBER papers, 10-Ks — the research corpus **never fits**.
- **The middle.** Even what fits is read unevenly: recall drops 20–30 pp. when evidence sits *mid-prompt* — the position–accuracy curve is U-shaped (Liu et al. 2024, *TACL*; Lec. 7 T5a). A $\sim 600\text{K}$ -token prompt also takes 30+ seconds to the first output token.

The December 2025 statement, concatenated after four years of minutes, lands mid-prompt — where the model reads worst. Cramming *hurts* quality, not just cost.

Economic Walls: The Meter and the Fresh Start

Even when pasting works once, it fails as a workflow: you re-pay the whole corpus on every question, and nothing accumulates.

- **Wall 3: the meter.** You pay for *every input token, every query* (per-token rates: Lec. 7 T4, “Tokens, Cost, and Reproducibility”). Stuff ~600K tokens of FOMC text into each query and one thousand exploratory queries re-read — and re-bill — the archive one thousand times. Prohibitive at the scale of a literature review.
- **Wall 4: the memory.** Naive prompting treats every query as a **fresh start**: no reuse of reading work across queries, no incremental update when next month’s minutes arrive, no way to grow past the window — ever.
- **Compare a real research workflow:** collect documents *once*; index them *once*; ask many questions over time, paying only for what you actually retrieve; index new arrivals incrementally.

The pattern we need: separate the cost of *ingesting documents* from the cost of *asking questions*. (The cost crossover gets its own picture in Act 8.4.)

The Wall

Line up what naive prompting can do against what research actually needs, and the gap is structural — not a matter of better prompts.

Naive prompt engineering can do	Research workflows actually need
QA on a single short document	QA over millions of tokens of corpus
Tens of queries before cost bites	Thousands of queries over weeks
Whatever fits in the context window	Decades of FOMC + NBER + 10-K filings
Today's prompt only	Incremental updates as new docs arrive
"Trust me" answers	Cited, source-attributed answers

Conclusion: prompt engineering hits a hard wall on freshness, scale, latency, cost, and attribution. We need an architecture that **separates indexing from querying**.

This is what RAG does.

The Human-Scale Fix: A Personal Wiki

Retrieval by hand — the pre-formal RAG

Before the Formalism: A Personal Wiki

Before the full machinery, meet the same idea at human scale — a knowledge base you can set up today.

- **Andrej Karpathy** (June 2025):

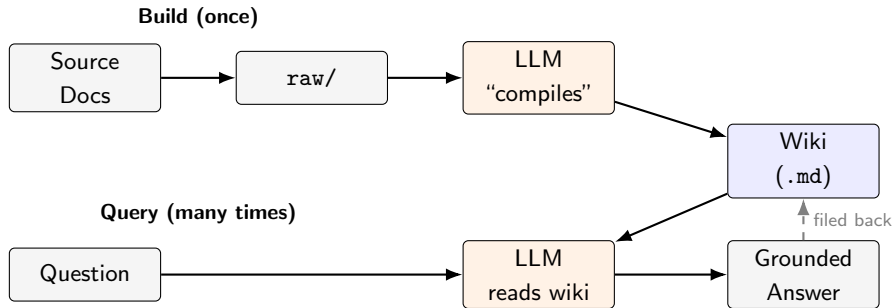
*"I'm finding very useful recently: using LLMs to build personal knowledge bases for various topics of research interest. [...] A large fraction of my recent token throughput is going less into manipulating code, and more into **manipulating knowledge** (stored as markdown and images)."*

- **The workflow is disarmingly simple:**

1. Collect source documents (papers, articles, data) into a `raw/` directory.
2. Ask an LLM to *compile* them into a **wiki** — a directory of `.md` files with summaries, concept articles, and cross-links.
3. Query the wiki by asking the LLM questions; it reads the relevant articles and answers from them.

- **Why this matters for us:** this is the *same core insight* behind RAG — separating data ingestion from querying — but without any of the formal machinery.

The Wiki Pipeline



- **Build:** papers, articles, and data land in `raw/`; the LLM compiles them into wiki articles — summaries, concept pages, back-links. *You rarely edit the wiki manually.*
- **Query:** ask a question; the LLM reads the wiki's index and relevant articles, then answers with grounded citations.
- **Feedback loop (dashed):** answers get filed *back* into the wiki, so your research “adds up.”

Hands-On: A Mini Wiki for This Course

A working example ships in this lecture's folder — browse it, extend it, query it.

```
wiki_demo/  
  index.md                -- master table of contents  
  concepts/  
    embeddings.md         -- text-to-vector mappings (Lec 7-8)  
    rag.md                -- retrieval augmented generation  
    transformers.md       -- the architecture behind LLMs  
  lectures/  
    lec07_llm.md          -- key ideas from Lecture 7  
    lec08_rag.md          -- key ideas from Lecture 8  
  papers/  
    vaswani_2017.md       -- Attention Is All You Need  
    lewis_2020_rag.md     -- RAG original paper (NeurIPS)  
  connections/  
    cross_references.md   -- links across lectures
```

- Ask: “How do the embeddings from Lecture 7 connect to vector stores in Lecture 8?” — the LLM reads the relevant pages and gives a **grounded, cross-referenced** answer.

Wiki = Manual RAG: Now Automate It

The wiki is retrieval, LLM-curated — and its ceiling tells us exactly what to build next.

	Personal Wiki	Full RAG Pipeline
Ingestion	LLM reads raw docs, writes summaries	Chunk → embed → store in vector DB
“Retrieval”	LLM reads index, picks articles	Embedding similarity search (top- K)
Scale limit	~400K words (index fits in context)	Millions of documents
Update	LLM rewrites/extends articles	Add new chunks incrementally

- **The natural progression:** naive prompting (dies at the window) → personal wiki (dies at ~400K words, when the LLM can no longer hold its own index) → **RAG** (embeddings + similarity search; scales to millions of documents).
- Karpathy again: *“the LLM has been pretty good about auto-maintaining index files [...] at this ~small scale.”* Thirty years of FOMC text is **not** small scale.

Bridge: From Hand-Curated to Machine-Indexed

Act 1 hit the wall: a frozen reader behind a metered window, a corpus that will never fit, and a hand-built wiki that fixes it only at human scale.

- RAG is the wiki idea *formalized and automated*: instead of an LLM maintaining an index by hand, embeddings and similarity search do retrieval at arbitrary scale. (Karpathy: “*there is room here for an incredible new product instead of a hacky collection of scripts.*”)
- **Next — Act 8.2 (INDEX):** the offline half of the pipeline. One picture of the whole machine, then its first two decisions: *chunking* (what is a retrieval unit?) and *embeddings* (text becomes geometry).
- Keep the two threads in sight: the FOMC minutes get chunked and embedded next; the OLG literature waits for Acts 8.3 and 8.5.

Arc: **Act 1 WALL** → Act 2 INDEX → Act 3 RETRIEVE → Act 4 GENERATE → Act 5 APPLY.

AI for Economic Research: Dynamic Models, Language, and Agents

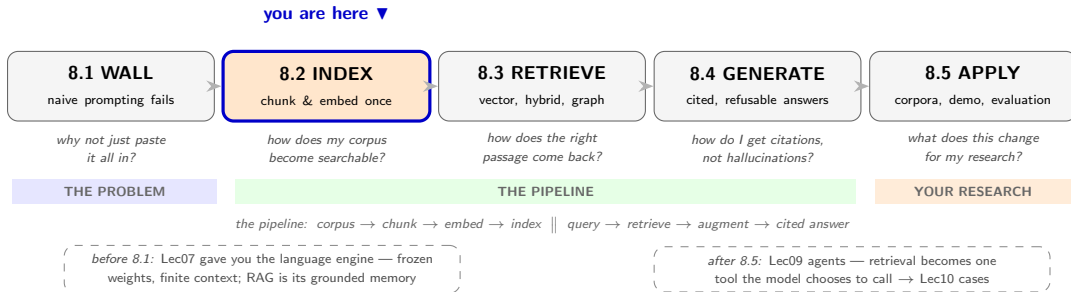
Lecture 8.2: Index — Chunking and Embeddings

Zhigang Feng

2026-07-08 07:39:04

Map: Act 8.2 — Build the Index

Act 1 hit the wall. Act 2 builds the machine's offline half: the index.



Route this deck: one pipeline picture → chunking (what is a retrieval unit?) → embeddings (text becomes geometry).
Thread A: the FOMC minutes get chunked and embedded here; Thread B: the same recipe indexes the OLG literature in Acts 8.3 and 8.5.

RAG: The Principled Solution

Index once, query many

The Key Insight: Index Once, Query Many

Don't paste the whole corpus into every prompt — pre-process it once, then fetch only what each question needs.

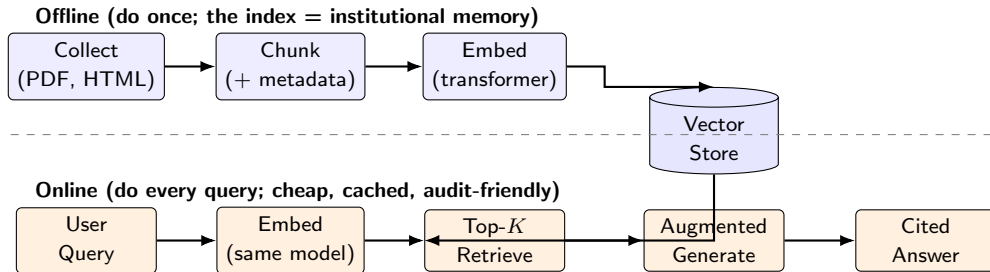
- **The recipe:**

1. **Pre-process documents once.** Split them into chunks, convert each chunk to a numerical fingerprint (an *embedding*), store everything in a database.
2. **At query time, fetch only what's relevant.** Convert the question to an embedding, look up the closest chunks, paste *those few chunks* into the prompt.
3. **Let the LLM answer from the retrieved evidence**, citing which chunks it used.

- **A two-phase architecture:** *offline*, index everything once; *online*, retrieve a few relevant pieces per query and generate.
- Cost is now proportional to *what you actually need*, not to the size of the corpus. Storage is amortized; indexes are updatable; the model only ever sees relevant text, so quality and faithfulness improve.

The Whole Pipeline in One Picture

The machine for the rest of the lecture — offline index built once, online loop run thousands of times.



Five parts: **chunker + embedder (this act)** → **vector store + retriever (8.3)** → **prompt + generator (8.4)**.

One rule already: use the *same embedder* for indexing and queries — otherwise the geometries don't line up.

Definition and Origin

Retrieval Augmented Generation (RAG): before answering, retrieve relevant documents from an external store and supply them to the model as context for generation.

- **Origin:** Lewis, Perez, Piktus, et al. (2020), “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS*.
- Originally a research architecture for open-domain QA. Now the *de facto* production pattern for any LLM application over a custom knowledge base: customer support, legal research, medical Q&A, code search — and academic research.
- **Key idea (worth repeating):** the LLM is the *reasoner*. The vector store is the *memory*. The retriever is the *librarian*. RAG is how we connect them.

Chunking

What is a retrieval unit?

Why Chunk At All?

Documents are too big to embed as a single vector — the corpus must be split into retrieval units first.

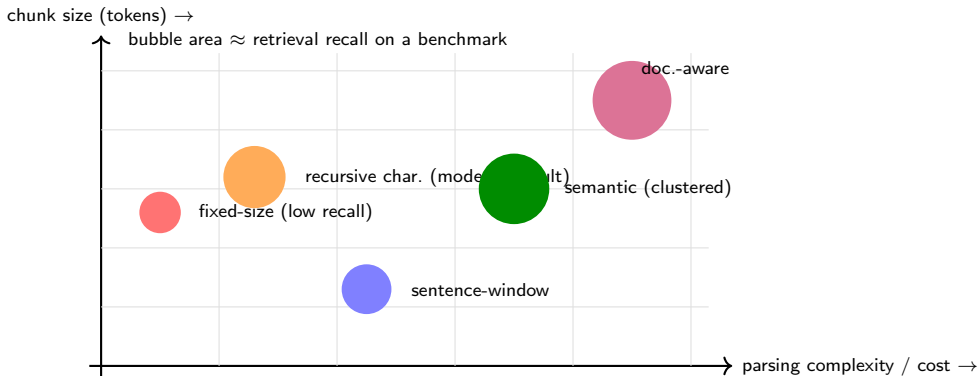
- **Three reasons to chunk:**

1. **Embedder input limits.** Most embedding models accept 512–8192 tokens per input. A 60-page FOMC transcript blows past every limit.
2. **Granularity of relevance.** Index a 30-page document as one vector and the embedding averages over many topics — the *one paragraph* on inflation expectations disappears into the mean.
3. **Generator context budget.** Only a few chunks go into the prompt. Smaller chunks $\Rightarrow$ more chunks fit $\Rightarrow$ richer evidence base.

- **Trade-off:** chunks too large $\Rightarrow$ noisy embeddings; too small $\Rightarrow$ context lost.

Choosing the chunking strategy is the most consequential decision in a RAG pipeline.

Chunking Strategies and the Trade-Off



Practical recommendation: start with *recursive character* (the LangChain default). Move up to *document-aware* when the corpus has rich structure (FOMC, 10-K, Congress). *Semantic chunking* — split where adjacent-sentence similarity drops — only when recursive retrieval is measurably noisy (it costs an embedding pass per sentence).

Fixed-Size vs. Recursive: An FOMC Example

The same paragraph, two chunkers — one destroys meaning, one preserves it.

- **Source paragraph** (FOMC minutes, simplified):

"Participants observed that inflation remained elevated. Several noted that core services prices, especially shelter, continued to rise. A few participants expressed concern about persistence in wage growth. Most agreed that policy should remain restrictive."

- **Fixed 60-character chunks:**

[■Participants observed that inflation remained elevat■]

[■ed. Several noted that core services prices, especia■] ...

⇒ **Cuts mid-word.** Embeddings of these fragments are nearly useless.

- **Recursive chunking** (split on paragraph → sentence → word, target ~120 chars):

[■Participants observed that inflation remained elevated.■]

[■Several noted that core services prices, especially shelter, continued to rise.■]

⇒ Each chunk is a self-contained sentence. Embeddings carry meaning.

Practical Defaults + Code

Three defaults cover most projects: chunk size 500–1000 tokens (~2–4 paragraphs); overlap 10–20%; **always preserve metadata** — doc ID, page, section, date, author.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,          # ~600 tokens
    chunk_overlap=100,       # ~80 tokens of overlap
    separators=["\n\n", "\n", ". ", " ", ""],
)

chunks = splitter.create_documents(
    texts=[fomc_minutes_text],
    metadatas=[{"meeting": "2024-06", "doc_type": "minutes"}],
)
```

The Economist's Chunking Gotcha

Real institutional documents have structure that generic chunkers destroy — and the structure is exactly what you will want to filter on later.

- **FOMC minutes:** organized as *speaker turns* (“Participants generally agreed. . .”, “A few members noted. . .”). Cutting across speaker boundaries fuses unrelated viewpoints.
- **Congressional Record:** hierarchical metadata — Congress / session / chamber / date / speaker / bill. Lose the hierarchy and you can’t filter or cite properly.
- **SEC 10-K filings:** standardized item numbers (Item 1, Item 1A Risk Factors, Item 7 MD&A). A query about “risk factors” should retrieve only *Item 1A* content — which only works if you parse the structure first.
- **Practical fix:** write a corpus-specific parser that emits chunks with rich metadata. Extra effort up front; a large payoff in retrieval quality and filtering (Act 8.3 uses exactly this metadata).

Don’t reach for the generic chunker. **Let your corpus tell you what a chunk should be.**

Embeddings

Text becomes geometry

From Text to Vectors: What and Why

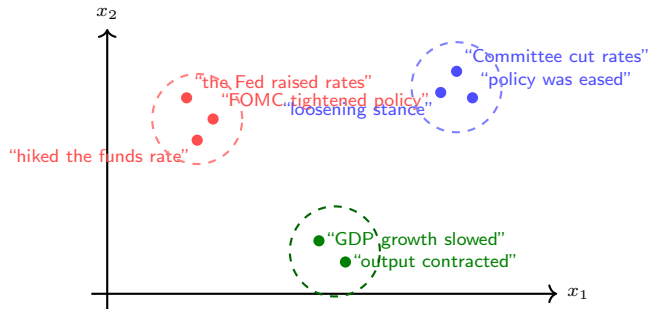
An embedding turns a piece of text into a point in space — and turns “find relevant passages” into geometry.

- An **embedding** is a function $E : \text{text} \rightarrow \mathbb{R}^d$ mapping text to a fixed-length vector; typical $d \in \{384, 768, 1024, 1536, 3072\}$.
- **The crucial property:**

semantically similar texts $\implies$ nearby vectors in $\mathbb{R}^d$

- **Why this is useful:** once the corpus lives in a vector space, “find documents about inflation expectations” becomes *nearest-neighbor search* around the query’s embedding.
- **Callback:** Lec. 7 T3b (“Token Embeddings vs. Document Embeddings — Bridge to Lec08”) showed token embeddings living *inside* the transformer. Here a separate *retrieval-tuned* model produces one vector per *chunk*.

Geometric Intuition: Trained So Similar Texts Land Nearby



- Hawkish clusters left; dovish right; growth talk elsewhere. **Distance in vector space $\approx$ distance in meaning** — “find me speeches like this one” becomes nearest-neighbor search under cosine (the hook Lec. 7 T2b planted).
- **Why it works:** a retrieval embedder is a transformer encoder *pooled* to one vector per text and *trained contrastively* — similar pairs pulled together, unrelated pushed apart — so the geometry is learned for similarity. *Sentence-BERT lineage: Reimers & Gurevych (2019); mechanics in Lec. 7 T3b.*

Choosing an Embedder (mid-2026)

Four models cover a researcher's realistic day-one choices — pick by hosting constraint, not leaderboard decimals.

Model	Dim	Access	Why pick it
OpenAI text-embedding-3-small	1536	API (metered)	cheap default, no infra
OpenAI text-embedding-3-large	3072	API (metered)	top accuracy, paid
voyage-3	1024	API (metered)	top retrieval benchmarks
bge-large-en-v1.5	1024	local (free)	best open-source default

Model facts current as of mid-2026.

For an academic researcher:

- **Local + free:** bge-large-en-v1.5 — runs on a laptop GPU; competitive with paid APIs.
- **Cloud + simple:** text-embedding-3-small — metered but inexpensive, zero infrastructure.
- **Top accuracy with budget:** voyage-3 or text-embedding-3-large.

Economists' Embedding Example: Fed Chair Speeches

Embed every Fed Chair speech 2010–2025, project to 2D, and the policy eras separate — with no labels supplied.

- **The setup.** Embed each speech (Bernanke, Yellen, Powell) with `bge-large-en-v1.5`; project to 2D with UMAP; plot.
- **What you see:**
 - Crisis-era Bernanke (2010–2014) clusters together — QE, financial stability.
 - Normalization-era Yellen (2015–2017) forms a distinct cluster — gradual exit, forward guidance.
 - Powell splits in two: 2020–2021 dovish “patient” speeches vs. 2022–2024 hawkish “restrictive” speeches.
- **Why this matters for research:** the clustering comes *purely from the text*. The economist gains a reproducible measure of stance shifts, linkable to rate decisions, market reactions, or macro outcomes.
- *Sources in this literature:* Hansen, McMahon & Prat (2018, QJE) — FOMC transcripts; Shapiro & Wilson (2021, REStat) — Fed sentiment; Gentzkow, Kelly & Taddy (2019, JEL) — text-as-data survey.

The Domain-Adaptation Problem

Generic embedders learned English from the open web — they did not learn Fed English.

- **Where they struggle:**

- “*patient*” in a Fed statement = *dovish* — a domain code word.
- CUSIP codes, BEA NIPA tables, FRED series IDs — alphanumeric strings with no web-English meaning.
- “Section 13(3)”, “LSAP” — technical citations the embedder has rarely seen.

- **Two fixes:**

1. **Fine-tune the embedder** on a domain corpus — a few GPU-hours (sentence-transformers, MultipleNegativesRankingLoss), real quality gains.
2. **Hybrid retrieval** (Act 8.3): combine dense embeddings with classical sparse retrieval (BM25), which catches the exact-match queries the embedder misses.

- **In practice:** hybrid is cheaper and gets you 80% of the gain. Fine-tune only when retrieval quality is demonstrably your bottleneck.

Code: Embed FOMC Chunks

```
from sentence_transformers import SentenceTransformer

# 1. Load a strong open-source embedder
model = SentenceTransformer("BAAI/bge-large-en-v1.5")

# 2. Suppose 'chunks' is a list of FOMC chunk strings
chunks = [
    "Participants observed that inflation remained elevated.",
    "Several noted that core services prices continued to rise.",
    # ... thousands more
]

embeddings = model.encode(    # one numpy vector per chunk
    chunks,
    batch_size=32,
    normalize_embeddings=True,    # so cosine sim = dot product
)

print(embeddings.shape)    # (N_chunks, 1024) -- ready for the store
```

Bridge: The Corpus Is Now Geometry

Act 2 built the offline half: chunks that respect the corpus's structure, carrying metadata, each with a learned vector.

- **Three load-bearing decisions made here:** chunk strategy (recursive vs. document-aware), embedder choice (open-source vs. API), domain adaptation (hybrid first; fine-tune only if retrieval bottlenecks).
- **Hands-on, two scales:** browse `wiki_demo/` (Act 8.1's manual wiki — markdown articles + an index, no retriever); then preview `demos/M4_rag_olg_demo` — a runnable *TF-IDF* retriever over an OLG survey paper, the sparse cousin of today's embeddings. Full classroom run in Act 8.5.
- **Next — Act 8.3 (RETRIEVE):** where the vectors live and how the right passage comes back — vector databases, similarity search, hybrid dense+sparse, metadata filters, and what to do when similarity alone is not enough (GraphRAG).

Arc: Act 1 WALL → Act 2 INDEX → Act 3 RETRIEVE → Act 4 GENERATE → Act 5 APPLY.

AI for Economic Research: Dynamic Models, Language, and Agents

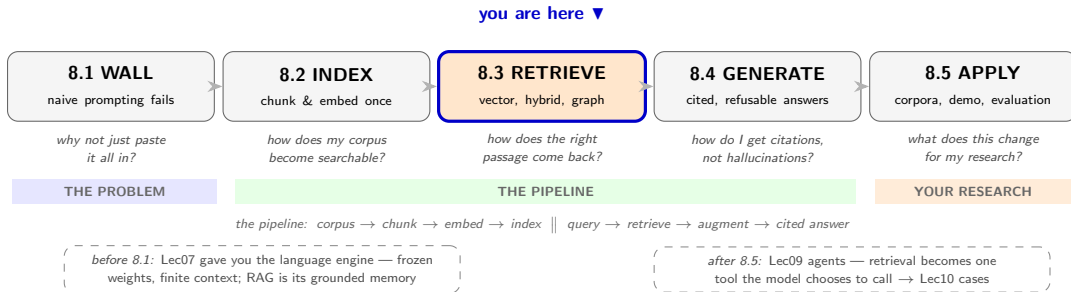
Lecture 8.3: Retrieve — Vector Search and GraphRAG

Zhigang Feng

2026-07-08 07:45:04

Map: Act 8.3 — Bring the Right Passage Back

The index exists; this act is the online lookup — and what to do when similarity alone is not enough.



Route this deck: vector search in practice → where similarity fails → GraphRAG and the GRAM case study. Thread A: filter FOMC chunks by date and type, then query them; Thread B: the OLG literature becomes a typed graph.

Vector Search in Practice

From similarity math to a working index

Why a Vector Database?

Millions of chunks now carry 1024-dim vectors — where do they live?

- **The naive option:** a flat numpy array on disk; search by computing similarity to every vector and sorting.
 - Works fine for 10,000 vectors; at 1,000,000 a query takes seconds; at 10,000,000 it is impractical.
 - No metadata filters. No incremental updates. No persistence.
- **A vector database** solves all four:
 1. *Scale*: approximate nearest-neighbor search over millions of vectors in milliseconds.
 2. *Metadata filters*: restrict search to “date $\geq$ 2020 AND doc_type = minutes.”
 3. *Incremental updates*: add new chunks as documents arrive, without rebuilding.
 4. *Persistence*: the index lives on disk and survives restarts.

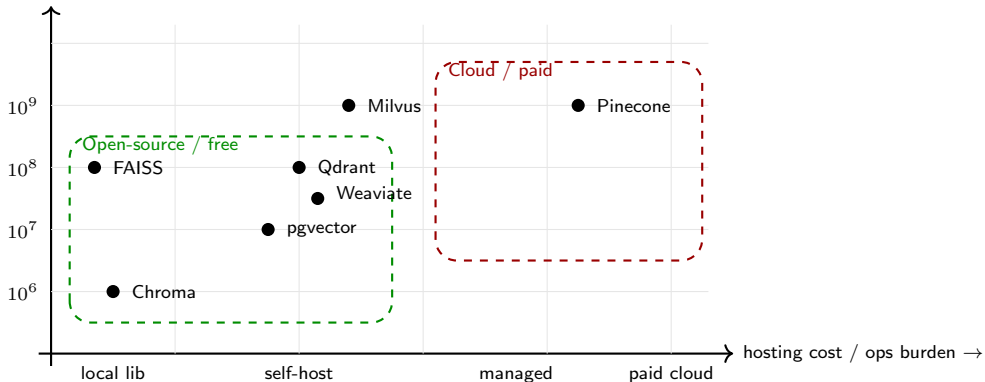
Similarity Search 101 — and Why Approximate

Retrieval is nearest-neighbor search: given a query vector q , find the K closest chunk vectors d_i .

- **Three common similarity functions:** cosine $\cos(\theta) = \frac{q \cdot d_i}{\|q\| \|d_i\|}$; dot product $q \cdot d_i$; (negative) Euclidean distance $-\|q - d_i\|$.
- **Why cosine is the default:** it ignores vector length. Modern embedders L2-normalize their outputs, so cosine = dot product: *higher dot product $\Rightarrow$ more similar*. Geometrically: each chunk is an arrow from the origin; cosine measures the *angle* to the query arrow.
- **Why approximate?** Exact search touches every vector: 10^7 vectors $\times$ 1024 dims $\approx 10^{10}$ operations *per query* — seconds of latency.
- **ANN (approximate nearest neighbor):** trade $\sim 1\%$ of recall for orders-of-magnitude speedups. Names to recognize when you read the docs: **HNSW** (graph walks) and **IVF** (cluster, then search the nearest cells). In practice *your vector DB picks the default* — you won't tune this by hand.

Vector DB Landscape: Scale vs. Hosting

practical scale (# vectors, log) →



For an economist starting out: Chroma or FAISS for laptop/cluster work (free, local, simple); **pgvector** if you already run Postgres (reuses existing infra); **Pinecone / Qdrant cloud** if you need a hosted index and have budget.

Hybrid Retrieval, Plus a Light Re-Rank

Dense retrieval finds meanings, not strings — so production systems pair it with a keyword engine and a re-ranking pass.

- **The dense-only failure case:** *“find every speech mentioning Section 13(3)”* — a precise legal-citation query. Embeddings cluster similar *meanings*, not similar *strings*; the top-10 neighbors discuss the discount window in general, not the section.
- **The fix — hybrid:** also run a classical sparse retriever in parallel. **BM25** (TF-IDF-based) scores exact term matches, catching rare strings, acronyms, statute citations, tickers. Merging the two ranked lists is a solved problem (*rank fusion*) that your library implements.
- **A light re-rank on top:** the fast bi-encoder returns a top-50 shortlist; a *cross-encoder* then reads query and candidate *together* and reorders — top-5 in, much sharper relevance out. Open default: `bge-reranker-large`.

Hybrid dense+sparse (+ re-rank) is the **production default** (mid-2026). Pure dense is rarely competitive.

Metadata Filters

Research queries carry structural constraints that pure semantic search cannot express.

- **Real example:** *“Find all FOMC discussions of unemployment from 2020 onwards, excluding press conferences.”*
 - “Discussions of unemployment” $\Rightarrow$ semantic search.
 - “From 2020 onwards” $\Rightarrow$ filter on `date` $\geq$ 2020-01-01.
 - “Excluding press conferences” $\Rightarrow$ filter on `doc_type` $\neq$ `press_conference`.

Modern vector DBs combine similarity with SQL-style filters in a single query.

- **This is exactly why preserving rich metadata at chunking time matters** (the Act 8.2 gotcha). If chunks don't carry `date`, `doc_type`, `speaker`, you cannot filter on them later.

Rule of thumb: add *more* metadata than you think you need. It is cheap at index time and impossible to reconstruct after the fact.

Code: Build a Chroma Index and Query It

```
import chromadb
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("BAAI/bge-large-en-v1.5")
client = chromadb.PersistentClient(path="./fomc_index") # on disk
coll = client.get_or_create_collection("fomc")

# Add chunks with embeddings + metadata
coll.add(
    ids=[f"chunk_{i}" for i in range(len(chunks))],
    documents=chunks,
    embeddings=model.encode(chunks, normalize_embeddings=True).tolist(),
    metadatas=[{"meeting": "2024-06", "doc_type": "minutes"} for _ in chunks],
)

# Query with a metadata filter
q = model.encode(["How did the Committee discuss inflation?"],
                 normalize_embeddings=True).tolist()
results = coll.query(query_embeddings=q, n_results=5,
                    where={"doc_type": "minutes"})
```

When Similarity Is Not Enough

Relationship questions need edges, not echoes

Where Vanilla Vector RAG Starts to Break

Dense retrieval is a local similarity machine — strong on lookup, blind to structure.

- It misses three important structures:
 1. **Relationships between objects.** Two chunks may matter because their entities are *linked*, even if the wording differs.
 2. **Global structure.** Nearest-neighbor search has no notion of themes, camps, or argument clusters across an entire corpus.
 3. **Coverage.** If the same fact appears in many chunks, top- K retrieval returns duplicates instead of a representative overview.
- **Economist examples:**
 - “Which Senators most often connect inflation to housing supply constraints?”
 - “What are the main camps in recent papers on LLMs and education?”
- Once the question is **multi-hop** or **corpus-level**, plain top- K chunk retrieval is often the wrong abstraction.

Concrete Break Point: The Phillips-Curve Question

Question: “Which Phillips-curve papers use survey expectations and are cited by post-2020 FOMC speeches?”

Vanilla vector RAG

- Embeds the query as one vector.
- Returns 5 chunks that *sound like* “Phillips curve + survey expectations.”
- Has **no notion of citation edges** between NBER papers and FOMC speeches.
- Best case: a chunk that *discusses* both topics, even if no citation link exists.
- Failure mode: confident, fluent, but the cited link is fictional.

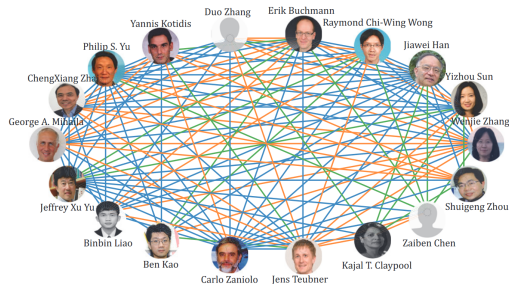
GraphRAG

- Walks the citation graph: NBER paper → cited_by edge → FOMC speech node.
- Filters by node attribute: date $\geq$ 2020.
- Filters by paper attribute: uses_data: SurveyOfProfessionalForecasters.
- Returns the actual subgraph; only then generates from the supporting chunks.
- Failure mode: degrades to “no path found,” not invention.

Rule: reach for GraphRAG when the question is *about a relationship*, not just *about a topic*. Top-*K* similarity cannot manufacture an edge that the index does not represent.

Knowledge Graphs: Adding Structure to Retrieval

- A **knowledge graph (KG)** stores facts as **nodes + edges** rather than isolated chunks.
- Canonical form: <head, relation, tail> — e.g. <Powell, discusses, inflation expectations>, or across papers: <Paper B, cites, Paper A>.
- **Build/use:** offline, extract entities and relations (optionally summarize communities); online, retrieve the *relevant subgraph* — entity nodes, chunk nodes, community summaries.
- The gain: retrieval is no longer “find text that sounds similar” — it can *follow links*.



Once facts are stored as a graph, retrieval can follow links rather than isolated paragraphs.

Vanilla RAG vs. GraphRAG: A Direct Comparison

Match the retrieval architecture to the question type — neither dominates everywhere.

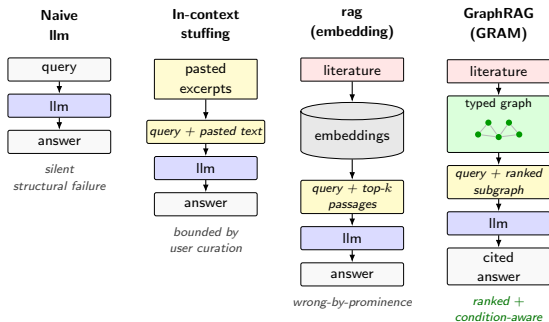
Question type	Vector / hybrid RAG	GraphRAG
Specific fact lookup	Strong when the exact chunk is nearby	Strong if fact is attached to an entity or path
Relationship query	Often requires several retrieved chunks and luck	Natural: traverse edges and retrieve neighborhoods
Global summarization	Top- K chunks under-cover the corpus	Community reports summarize modules of the corpus
High redundancy corpus	May retrieve near-duplicate boilerplate	Can collapse repeated facts into entities/communities
Cost and complexity	Simpler, cheaper, easier to debug	More expensive offline; KG quality matters

Research rule: start with vector + BM25 + re-rank. Move to graph retrieval *only* when the question requires relationships or corpus-level themes you can't get from local similarity.

Case Study: GRAM — Four Mechanisms on One OLG Query

The instructor's own GRAM: a domain-typed GraphRAG over a curated 157-paper OLG corpus — the research-grade sibling of the Act 8.5 demo.

"OLG life-cycle, idiosyncratic + aggregate shocks, incomplete intergenerational asset markets — which solver class is admissible for the joint stationary distribution in recursive equilibrium?"



Why economics is special: every paper is math and prose — math is analysis, prose is mechanism. An econ assistant needs both; GRAM sits between math-IR and legal/clinical RAG.

Wrong-by-Prominence vs. Condition-Aware Retrieval

- **Wrong-by-prominence:** pure embedding retrieval routes the query to the *most prominent* paper, whether or not it fits the asked-for conditions:

- “OLG with capital” $\Rightarrow$ Diamond (1965), *not* Auerbach–Kotlikoff (1987).
- “heterogeneous-agent macro” $\Rightarrow$ Aiyagari, *not* Krusell–Smith.

The vocabulary is present; the applicability filter is absent.

- **Condition-aware:** admissibility is *conditional* — add aggregate risk and a different solver class becomes admissible. Embedding similarity cannot encode that dependency; a walk over a *typed* graph of model objects can.
- **Two corpus lessons worth copying:** a non-trivial *post-cutoff share* (RAG demonstrably adds value); *citation connectivity* (unlinked documents are unreachable).

Honest framing. These labels name each mechanism's *failure mode* on this query type. GRAM's condition-aware advantage is the **gap a planned three-condition evaluation (naive / RAG / GraphRAG) is designed to measure** — a design goal, not yet a reported win; one in-progress, unpublished pipeline.

Choosing the Retrieval Architecture

Question type	Best first choice	Why
Precise factual lookup or citation hunt	Hybrid dense + sparse RAG	Exact strings, dates, and citations matter more than global structure.
Specific multi-hop entity question	Graph-based RAG	The answer depends on linked entities and paths rather than one chunk.
Broad summarization over a large corpus	Graph / hierarchical retrieval	You need coverage of themes, not just the closest paragraphs.
Fast prototype on a changing corpus	Vanilla RAG first	A flat vector index is easy to build and update; add structure only if it buys accuracy.

Rule of thumb: start with hybrid dense+sparse. Escalate to graph retrieval when the task requires *relationships*, *coverage*, or *multi-hop reasoning*.

Names to recognize in this space: Microsoft GraphRAG, LightRAG, HippoRAG, RAPTOR — Edge et al. (2024); Guo et al. (2024); Gutiérrez et al. (2024); Sarthi et al. (2024).

Bridge: The Right Passage Is Back — Now Answer From It

Act 3 delivered the online lookup: hybrid vector+keyword search with metadata filters as the default, graph retrieval when the question is about relationships.

- The GRAM case shows where this is heading for research corpora: typed structure over the literature, so retrieval respects *conditions*, not just vocabulary.
- Eventually an *agent* will choose vector vs. graph vs. web search itself — Act 8.5 closes on that bridge to Lecture 9.
- **Next — Act 8.4 (GENERATE):** retrieved chunks are not an answer yet. The generation contract: a prompt template that grounds, cites, and *refuses* — plus the decision matrix against fine-tuning.

*Arc: Act 1 WALL → Act 2 INDEX → **Act 3 RETRIEVE** → Act 4 GENERATE → Act 5 APPLY.*

AI for Economic Research: Dynamic Models, Language, and Agents

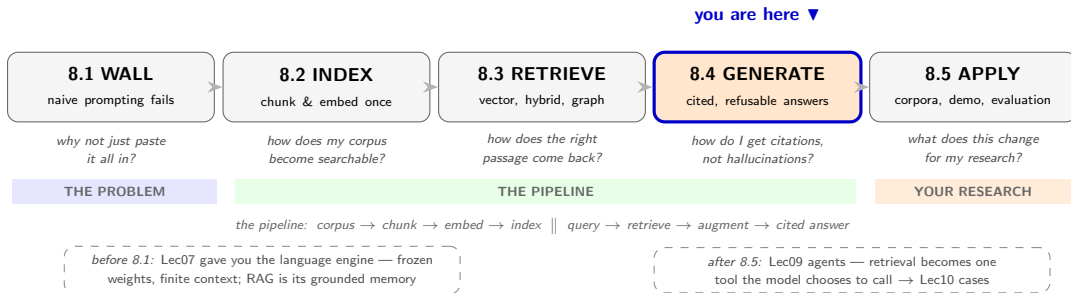
Lecture 8.4: Generate — Grounded Answers and RAG vs Fine-Tuning

Zhigang Feng

2026-07-08 07:49:33

Map: Act 8.4 — From Evidence to a Cited Answer

Retrieval hands back passages; this act turns them into an answer you can put in a paper.



Route this deck: the generation contract (ground, cite, refuse) → a worked FOMC run → the decision matrix vs fine-tuning. Thread A carries the worked example; Thread B returns in the Act 8.5 classroom demo.

Grounded Generation

From retrieved evidence to a cited, refusable answer

From Retrieved Chunks to a Cited Answer

The generation contract: the answer must be grounded in the retrieved evidence, cited chunk by chunk, and refusable when the evidence runs out.

- We now have the top- K chunks for the query (typically $K = 3$ to 10). **Insert them into a prompt template and send to the LLM.** The model sees:
 1. System instructions (how to behave)
 2. The retrieved chunks (the evidence)
 3. The user question
 4. Output-format instructions (how to cite)
- This is **the only place a generative model is used** in the whole pipeline. Everything before — chunking, embedding, retrieval — exists to put high-quality evidence in front of the model.
- **The single most important design choice** at this stage: the *prompt template*.

Prompt Template — Anatomy

SYSTEM:

You are a research assistant for academic economists.
Answer the user's question using ONLY the context below.
If the context does not contain the answer, reply exactly: "I don't know based on the provided sources."
Cite sources inline using [doc_id:page] notation.

CONTEXT:

[1] (fomc_2024_06:p3)

"Participants observed that inflation remained elevated..."

[2] (fomc_2024_06:p4)

"Several noted that core services prices, especially shelter, continued to rise..."

QUESTION:

How did the FOMC discuss inflation expectations in June 2024?

INSTRUCTIONS:

Answer in 2-3 sentences. Cite each claim inline as [doc_id:page].

Four parts: system contract, evidence, question, format — the contract lines do the grounding work.

Source Attribution Mechanics

Every claim in an academic paper must be traceable to a source — an ungrounded LLM output is not citable; a grounded output is.

- **How to enforce:**

1. At chunking time, give every chunk a stable `doc_id` and `page` (or `turn`, or `section`).
2. At retrieval time, return the metadata along with the chunk text.
3. In the prompt template, label each chunk `[1] (doc_id:page)` and instruct the model to cite using that label.
4. In post-processing, replace inline citations with hyperlinks back to the original document.

- **Before / after:**

- *Ungrounded*: "The FOMC was concerned about inflation persistence in 2024."
- *Grounded*: "The FOMC was concerned about inflation persistence in mid-2024 [fomc_2024_06:p3], particularly in core services prices including shelter [fomc_2024_06:p4]."

Refusal and Faithfulness

“I don’t know” is a feature, not a bug: in research, a fabricated answer is far worse than a refusal.

- **Faithfulness** (every claim supported by the retrieved context) matters more than *fluency*. A fluent fabrication is the worst possible outcome for research.
- **How to enforce refusal:**
 - Explicit system-prompt instruction: “If the context does not contain the answer, reply exactly: I don’t know based on the provided sources.”
 - Low generation temperature (0.0–0.2) to discourage creative inference.
 - In post-processing, run a faithfulness check (Act 8.5) and flag unsupported claims.
- **Example refusal (good behavior):**

“I don’t know based on the provided sources. The retrieved minutes from June 2024 do not discuss the impact of fiscal policy.”

This is what protects the researcher from quoting fiction.

Worked Example: Full Pipeline Run

Query. *“How did Powell discuss inflation expectations in 2023 press conferences?”*

Pipeline trace:

1. **Embed query** with bge-large-en-v1.5 → 1024-d vector.
2. **Retrieve top-50** via Chroma + BM25 hybrid. Filter: doc_type = press_conference AND year = 2023.
3. **Re-rank** the 50 with bge-reranker-large → top-5.
4. **Inject top-5 chunks** into the prompt template (previous slide), tagged [powell_2023_03:p4], [powell_2023_06:p2], ...
5. **Send to the generator LLM** at temperature=0.1. Receives:
“In 2023 press conferences, Chair Powell repeatedly emphasized that long-run inflation expectations remained well anchored [powell_2023_03:p4], while noting that short-run expectations had risen with realized inflation [powell_2023_06:p2]. By year-end, he highlighted that survey- and market-based measures had begun to ease [powell_2023_12:p3].”
6. **Post-process:** convert inline citations to clickable links back to the transcript pages.

RAG vs. Fine-Tuning

Facts need retrieval; style needs weights

Decision Matrix: RAG vs. Fine-Tuning

Dimension	RAG	Fine-tuning
Knowledge freshness (update daily)	strong	weak
Source attribution / citations	strong	none
Reduces hallucination	strong	limited
Cost to add new documents	near zero	re-train
Latency at query time	moderate	low
Style adaptation ("write like the Fed")	weak	strong
Output-format adaptation (structured JSON)	limited	strong

Reading the matrix (green = native strength, yellow = workable, red = fundamental weakness): RAG dominates on the four dimensions that matter most for *research* — freshness, attribution, hallucination control, update cost. Fine-tuning wins on style and format, which matter for *production* but rarely for research.

When Fine-Tuning Still Makes Sense

Fine-tuning earns its keep on style, format, and jargon — not on facts.

- **Style transfer.** You want the model to *write like a Federal Reserve economist* — cadence, hedging, phrasing. Fine-tune on a thousand FOMC-minutes examples. RAG can't teach style.
- **Structured-output formatting.** Always emit a JSON dictionary of DSGE parameters? Fine-tune on ~ 500 input/output pairs.
- **Specialized vocabulary.** Teach the model that LSAP = Large-Scale Asset Purchase, NIPA = National Income and Product Accounts.
- **Latency-critical settings.** No retrieval round-trip — useful in real time, rarely binding for research.

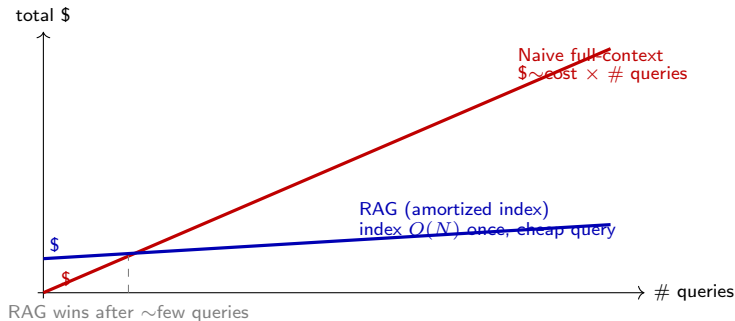
Crucial caveat: fine-tuning $\neq$ teaching new *facts*. Models memorize facts unreliably during fine-tuning and still hallucinate. *If you want the model to know new facts, use RAG* — Lec. 7 T5b's closing rule: fine-tuning stores *style*, not *facts*.

The Hybrid Pattern

In production the modern answer is both, in different roles: **fine-tune the small model**, **retrieve for the big one**.

- **Fine-tune the embedding model** on a domain corpus (say, 100K FOMC sentences with synthetic query–document pairs). Cheap, and it improves retrieval quality dramatically.
- **Use a general-purpose frontier LLM as the generator** — no fine-tuning of the big model.
RAG is the glue: the fine-tuned embedder finds the right chunks; the general LLM writes fluent, faithful prose from them.
- **Why this works:** domain understanding from the (small) fine-tune, freshness + attribution from RAG — without the cost or fragility of fine-tuning a frontier-scale generator.
- The dominant production architecture (mid-2026) for LLM applications over specialized corpora — legal, medical, financial, academic.

Cost: Index Once, Query Many (Visual)



- **Naive full-context prompting:** cost scales *linearly* with context size *and* $\#$ queries — every question repays the full corpus token bill (Act 8.1's meter wall).
- **RAG:** pay $O(N)$ embedding calls *once* at index time; each query ships only the top- K chunks ($\sim 1\text{--}5\%$ of the corpus). Marginal cost per query collapses.
- **Per-token rates and the reproducibility tax:** Lec. 7 T4, "Tokens, Cost, and Reproducibility." Past a handful of queries, index-once dominates on total spend *and* latency.

Bridge: The Pipeline Is Complete — Now Make It Earn Its Keep

Act 4 closed the loop: retrieved chunks become cited evidence, the generator is contractually forced to ground, cite, and refuse — and the decision matrix says when to fine-tune instead.

- **What changed:** the output is now *auditable*. That is what makes RAG a research tool rather than a chat trick.
- **Rule to remember:** style, format, jargon → fine-tune; facts → retrieve.
- **Next — Act 8.5 (APPLY):** the payoff. Six research corpora end to end, a classroom OLG demo you can run offline today, the six failure modes and their mitigations, RAGAS + human validation — and the bridge to Lecture 9's agents.

Arc: Act 1 WALL → Act 2 INDEX → Act 3 RETRIEVE → Act 4 GENERATE → Act 5 APPLY.